1 Introduction

1.1 Problem statement

The main goal is to estimate the arboricity in a graph G $\lambda(G)$ - a metric that describes the minimal number of edge-disjoint forests that G can be split into. Existing algorithms can give the exact answer in $O(n+m)$ time, where here and later $|V(G)| = n, |E(G)| = m$, and different degree approximations in sublinear time: O(1) approximation in $O(\frac{n}{\lambda})$by Dai et al.[1], 2 approximation in O()[**?**] and TODO.

1.2 Graph input model

The graph is given in form of a adjacency list, where we can in O(1) acess the i-th neighbour of the j-th vertex, and get the vertex degree by counting the number of neighbours.

1.3 Graph metrics

Main graph metrics that we will consider later are following:

$$\rho(G) = max_{S \subseteq V(G)} \frac{|E(S)|}{|S|} \text{ - fractional density;}$$

$$\gamma(G) = max_{S \subseteq V(G), |S| \geq 2} \frac{|E(S)|}{|S|-1} \text{ - fractional arboricity;}$$

$$\lambda(G) = max_{S \subseteq V(G), |S| \geq 2} \lceil \frac{|E(S)|}{|S|-1} \rceil \text{ - integer arboricity;}$$

,

We can derive relations between them:

$$\lambda(G) = \lceil \gamma(G) \rceil$$

$$\gamma(G) - \rho(G) \leq \frac{|E(S)|}{|S|-1} - \frac{|E(S)|}{|S|} = \frac{E(S)}{|S|(|S|-1)} \leq \frac{1}{2}$$

$$\text{because } |E(S)| \leq \frac{|S|(|S|-1)}{2} \text{(maximum number of edges when S is a clique)}$$

Therefore by estimatimating the fractional density with factor $(1+\varepsilon)$ we can approximate integer arboricity like $\lambda(G) = \lceil \gamma(G) \rceil \leq \lceil \rho(G) + \frac{1}{2} \rceil \leq \lambda(G)(1+\varepsilon) + 1$.

2 Algorithm

2.1 Decision problem formulation

Simillarly to Dai et al.[**?**] we formulate arboricity approximation scheme as the decision problem by using a comparator $Test(G, \rho, \varepsilon)$ that takes graph G, approximation constant

$\varepsilon \in (0; 0.5)$, and current approximation of the fractional density rho^* as an input. We iterate over $\rho_i := \frac{n}{(1+\varepsilon)^i}$ from $i = 0$ while $\rho_i > \frac{1}{n}$. On the i-th iteration we run $Test(G, \rho_i, \varepsilon)$, it constructed in such way that it returns YES if $\rho(G) \leq \rho_i$ and NO if $\rho(G) \geq (1+\varepsilon)\rho_i$.

We continue taking iterations until first NO verdict and return approximation as the $\lceil \rho_i + \frac{1}{2} \rceil$ on the last ρ_i with YES.

2.2 Test() details

Algorithm of the $Test(G, \rho_i, \varepsilon)$ can be split into following 3 simple steps:

1. **High density check**: If number of edges in a graph is already greater than $\rho_i |V(G)|$, then we can return NO straight away and go to the next iteration;
2. **Construction of the sparse graph H based on G**: We get sparse graph H with expected $O(n \log n)$ number of edges by sampling them from graph G;
3. **Fractional density estimation in H**: We use Multiplicative weights update(MWU) scheme, initially proposed by TODO[**?**], to solve fractional edge orientation LP and in the end get $(1+\varepsilon)$ approximation of the maximal outdegree in a graph with minimal orientation. If $D' \leq (1+\varepsilon_1)((1+\varepsilon_2)\lambda p)$, $((1+\varepsilon_2)\lambda p$ is threshold maximum outdegree in H) return YES, else NO;

High level depiction of the algorithm can be seen on the Figure 1.

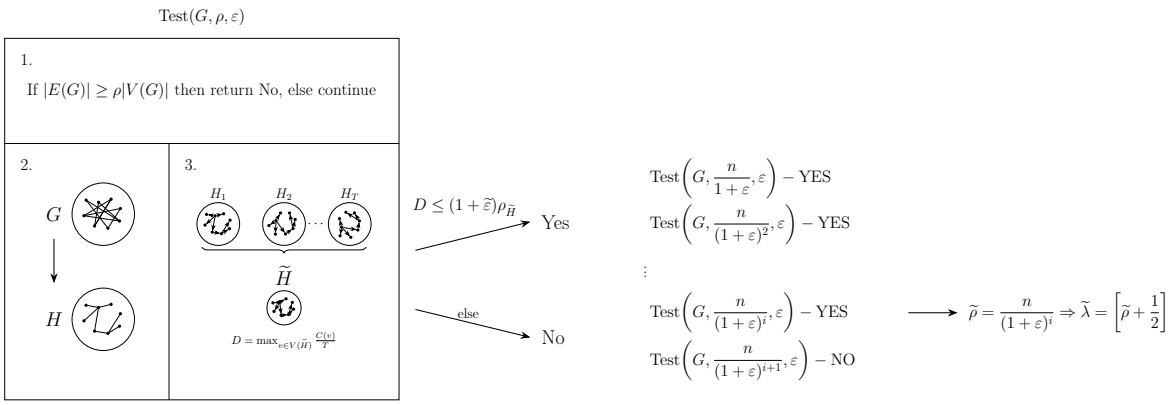

Figure 1: High level scheme of the proposed algorithm

2.2.1 High density check

In $O(n)$ time we can get the sum of all of the vertex degrees in a graph that is equal to the doubled number of edges in it $2m$. If found $m \geq \rho_i n$, then by result of [**?**] there must be a $\rho densecorethatbringustotheimmideateNOanswer$.

2.2.2 Construction of the sparse graph H based on G

The main goal of this step is to acquire a sparce approximation H of the original graph G in $\widetilde{O}$ time and query complexity.

2.2.3 Fractional density estimation in H

G has a maximum subgraph density D if and only if you can orient the edges of G such that no vertex has an out-degree strictly greater than D. This step gives $(1+\varepsilon)$ approximation of D. [?]

3 Correctness and error bound estimation

3.1 Yes case

By construction of the algorithm we only get yes if we pass the step 1 and then get the approximation of the D in step 3 that is lower that the set threshold.

Resulting error of geting false positive result then is

todo

3.2 No case

By the construction of the algorithm we get NO result either at the step 1 where we discover the trivial high density of the entire graph G that was already discussed earlier or during the step 3 when estimated D turned out to be greater than the threshold.

Resulting error of getting false negative result is

todo

4 Asymptotic analysis

4.1 Time complexity

By utilising the stepping scheme of the algorithm we take a number of iterations to get the first NO verdict, we can estimate the number of iterations taken as $\log_{(1+\varepsilon)} n = O(\frac{\log n}{\varepsilon})$ that becomes a multiplicative factor that contributes to the overall time complexity.

Step 1 of the Test() takes $O(n)$ queries that run in $O(1)$ time from the model of input graph that we use. These queries are used to get degres of all of the vertices and get the doubled number of edges in G. Step 2 takes up $O(n \log n)$ queries judging from the resulting expected number of edges in a graph, where each requires $O(1)$ time complexity query.

Resulting time complexity $O(n\ poly(\log n)) = \widetilde{O}(n)$

4.2 Query complexity

Resulting query complexity $O(n\ poly(\log n)) = \widetilde{O}(n)$

4.3 Additional space

Additional space is used to store the sparse graph H with $O(n \log n)$ expected edges and n nodes that results in $\widetilde{O}(n)$ additional space used.

References

[1] Jiangqi Dai, Mohsen Ghaffari, and Julian Portmann. Constant approximation of arboricity in near-optimal sublinear time. *arXiv preprint arXiv:2512.18416*, 2025.